Notes of Modern Policy Learning Methods

Yue Wu

September 23, 2026

Outline

Intro
- Measure a policy space

Moving Towards PPO
- Intro
- (2015) Trust Region Policy Optimization (TRPO)
- (2015) Generalized Advantage Estimation (GAE)
- (2016) Advantage Actor-Critic (A2C)
- (2017) Proximal Policy Optimization (PPO)

LLM fine-tuning
- Intro
- (2022) Reinforcement Learning with Human Feedback (RLHF)
- (2023) Direct Policy Optimization (DPO)
- (2024) Group Relative Policy Optimization (GRPO)
- (2024) On-Policy Distillation (OPD)

Beyond MDP
- (2021) Decision Transformer

Discussion

Outline

- Intro
  - Measure a policy space
- Moving Towards PPO
  - Intro
  - (2015) Trust Region Policy Optimization (TRPO)
  - (2015) Generalized Advantage Estimation (GAE)
  - (2016) Advantage Actor-Critic (A2C)
  - (2017) Proximal Policy Optimization (PPO)
- LLM fine-tuning
  - Intro
  - (2022) Reinforcement Learning with Human Feedback (RLHF)
  - (2023) Direct Policy Optimization (DPO)
  - (2024) Group Relative Policy Optimization (GRPO)
  - (2024) On-Policy Distillation (OPD)
- Beyond MDP
  - (2021) Decision Transformer
- Discussion

Intro (working)

This note mainly focus on some modern Policy Learning methods, including

- Policy based method
  - TRPO
  - PPO
  - ...
- Decision Transformer (Without MDP assumption)

RL-based view.

KL divergence

Two views of optimizing a policy-like-function.
Optimizing a Function v.s. Optimizing a Policy in RL sense

Outline

Intro
- Measure a policy space

Moving Towards PPO
- Intro
- (2015) Trust Region Policy Optimization (TRPO)
- (2015) Generalized Advantage Estimation (GAE)
- (2016) Advantage Actor-Critic (A2C)
- (2017) Proximal Policy Optimization (PPO)

LLM fine-tuning
- Intro
- (2022) Reinforcement Learning with Human Feedback (RLHF)
- (2023) Direct Policy Optimization (DPO)
- (2024) Group Relative Policy Optimization (GRPO)
- (2024) On-Policy Distillation (OPD)

Beyond MDP
- (2021) Decision Transformer

Discussion

Intro: Moving Towards PPO

- REINFROCE
- Trust Region Policy Optimization (TRPO)
  - can be seemed as a smart-conservative-greedy algorithm by **bounding the difference between old and new policy**
  - theoretically good but hard to implement (solving a constrained optimization problem is always hard)
- Proximal Policy Optimization (PPO)
  - similar goal as TRPO, but use a pessimistic bound, **clip on the goal directly instead of using KL-divergence constraint**
  - compared to TRPO, a little bit ad hoc, but more technically efficient, easy to implement
- Some algorithm refinement
  - GAE
  - A2C

Preliminaries: REINFORCE

(Williams 1992)(Sutton et al. 1999)

Recall Policy Gradient Theorem and REINFORCE with baseline,

$$\nabla_\theta V^{\pi_\theta}(s_1) = \mathbb{E}\left[Q^{\pi_\theta}(s,a)\nabla_\theta \log(\pi_\theta(s,a))\right]$$

$$= \mathbb{E}\left[\underbrace{\left(Q^{\pi_\theta}(s,a) - V^{\pi_\theta}(s)\right)}_{A(s,a)} \nabla_\theta \log(\pi_\theta(s,a))\right]$$

where

$$\mathbb{E} = \mathbb{E}_{s\sim d^{\pi_\theta}(s_1), a\sim\pi_\theta(\cdot|s)}$$

Its sample estimate is

$$\hat{R}_t^{(i)} := \sum_{t'=t}^{T-1} \gamma^{t'-t} r_{t'}^{(i)}$$

$$\hat{\mathbf{g}} = \frac{1}{N}\sum_{i=1}^{N}\sum_{t=1}^{T} \gamma^{t-1} \underbrace{\left(\hat{R}_t^{(i)} - b_t(s_t^{(i)})\right)}_{\hat{A}_t} \nabla_\theta \log(\pi_\theta(s_t^{(i)}, a_t^{(i)}))$$

For RL basics, see `https://oftrongs.github.io/RL/RL_summary.pdf`

Trust Region Policy Optimization (TRPO)

Introduce KL as policy distance measure(Schulman, Levine, et al. 2015)

TRPO extends Policy Improvement Theorem by replacing α with a distance measure between π and $\tilde{\pi}$.

$$\alpha \leftarrow D_{TV}^{max}(\pi, \pi^{new})$$

This theorem quantifies quantifies the least change of value for any two policies $\pi, \tilde{\pi}$ with policy advantage and KL-divergence of the two policies,

$$\eta(\tilde{\pi}) - \eta(\pi) \geq A_{\pi}(\tilde{\pi}) - \frac{2\gamma\varepsilon}{(1-\gamma)^2} D_{KL}^{\max}(\tilde{\pi}, \pi)$$

So we can design an Approximate Policy Iteration algorithm that guarantees increasing value:

In each iteration k = 0,1,2,... Solve the constrained optimization problem

$$\pi_{k+1} = \arg\max_{\pi} \left[A_{\pi_k}(\pi) - \frac{2\gamma\varepsilon}{(1-\gamma)^2} D_{KL}^{\max}(\pi_k, \pi) \right]$$

Trust Region Policy Optimization (TRPO)

Introduce KL as policy distance measure(Schulman, Levine, et al. 2015)

Now consider parameterized policies π_θ, and uses a constraint on the KL divergence rather than a penalty[1].

In each iteration $k = 0, 1, 2...$

$$\theta_{k+1} = \begin{array}{ll} \arg\max_\theta & \mathbb{E}_{s\sim\pi_{\theta_k}}\left[\sum_a \pi_\theta(s,a) A^{\pi_{\theta_k}}(s,a)\right] \\ \text{s.t.} & \mathbb{E}_{s\sim\pi_{\theta_k}}\left[D_{\mathrm{KL}}(\pi_\theta(\cdot|s), \pi_{\theta_k}(\cdot|s))\right] \leq \delta \end{array}$$

In practice, use the sample-based estimation of $\mathbb{E}_{s\sim\pi_{\theta_k}}$ and $A^{\pi_{\theta_k}}$ and the sample-based estimation of average KL-divergence.

$$\theta_{k+1} = \begin{array}{ll} \arg\max_\theta & \hat{\mathbb{E}}_{s\sim\pi_{\theta_k}}\left[\sum_a \pi_\theta(s,a) \hat{A}^{\pi_{\theta_k}}(s,a)\right] \\ \text{s.t.} & \hat{\mathbb{E}}_{s\sim\pi_{\theta_k}}\left[D_{\mathrm{KL}}(\pi_\theta(\cdot|s), \pi_{\theta_k}(\cdot|s))\right] \leq \delta \end{array}$$

[1] In practice, if using the penalty coefficient $\frac{2\gamma\varepsilon}{(1-\gamma)^2}$, the step sizes would be very small

Generalized Advantage Estimation (GAE)

(Schulman, Moritz, et al. 2015)

Advantage Actor-Critic

get trajectories/reward in parallel

$$\text{Recall } \nabla_\theta V^{\pi_\theta}(s_1) = \mathbb{E}\left[\underbrace{(Q^{\pi_\theta}(s,a) - V^{\pi_\theta}(s))}_{A(s,a)} \nabla_\theta \log(\pi_\theta(s,a))\right],$$

In actor-critic method, the policy $\pi(\cdot,\cdot)$ is the actor and the baseline $V^\pi(\cdot)$ is the critic.

The question is: How we estimate this gradient, or How we estimate this Advantage?

A2C estimates this value through getting the trajectories by multi-agents using same policy in copy-worlds. This can efficiently get large sample, and we get a more robust estimate in statistical sense.

> "possible with both value-based and policy-based methods, off-policy as well as on-policy methods, and in discrete as well as continuous domains." (Mnih et al. 2016)

Proximal Policy Optimization (PPO)

(Schulman, Wolski, et al. 2017)

Let $p_k(\theta) := \frac{\pi_\theta(s,a)}{\pi_{\theta_k}(s,a)}$, recall that TRPO maximize (under constraint)

$$\theta_{k+1} = \arg\max_\theta \hat{\mathbb{E}}_{s,a\sim\pi_{\theta_k}} \left[\frac{\pi_\theta(s,a)}{\pi_{\theta_k}(s,a)} \hat{A}^{\pi_{\theta_k}}(s,a) \right],$$

$$L^{CPI}(\theta) = \hat{\mathbb{E}}_k \left[p_k(\theta) \hat{A}_k \right]$$

But TRPO is hard to compute(a constrained optimization problem), so an ad-hoc but efficient way comes:

PPO instead solve the unconstrained problem by maximize

$$L^{CLIP}(\theta) = \hat{\mathbb{E}}_k \left[\min \left(p_k(\theta)\hat{A}_k, \text{clip}(p_k(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_k \right) \right]$$

Why clip-then-min?

a pessimistic bound (Schulman, Wolski, et al. 2017)

$$L^{CLIP}(\theta) = \hat{\mathbb{E}}_k \left[\min \left(p_k(\theta)\hat{A}_k, \text{clip}(p_k(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_k \right) \right]$$

- a pessimistic bound in the sense that
  - clip(ignore) when the policy improve
    - if $\hat{A} > 0$ and $p_k > 1+\epsilon$, the final is $(1+\epsilon)\hat{A}$
    - if $\hat{A} < 0$ and $p_k < 1-\epsilon$, the final is $(1-\epsilon)\hat{A}$
  - unclip when policy is worse
    - if $\hat{A} > 0$ and $p_k < 1-\epsilon$, the final is $p_t\hat{A}$
    - if $\hat{A} < 0$ and $p_k > 1+\epsilon$, the final is $p_k\hat{A}$

A soft way of conservative-greedy. By maximizing a pessimistic lower bound on the surrogate objective, PPO avoids the large jump to an estimated-better but actually-worse policy.

Proximal Policy Optimization (PPO)

(Schulman, Wolski, et al. 2017)

Algorithm 1 PPO, Actor-Critic Style

for iteration=$1, 2, \ldots$ **do**
  for actor=$1, 2, \ldots, N$ **do**
    Run policy $\pi_{\theta_{\text{old}}}$ in environment for T timesteps
    Compute advantage estimates $\hat{A}_1, \ldots, \hat{A}_T$
  end for
  Optimize surrogate L wrt θ, with K epochs and minibatch size $M \leq NT$
  $\theta_{\text{old}} \leftarrow \theta$
end for

Number of actors N, rollout length T, optimization epochs K, minibatch size $M \leq NT$, discount factor γ, GAE parameter λ, clipping parameter ϵ, coefficients c_V, c_H.

Algorithm 1 Compute advantage estimates from $\pi_{\theta_{\text{old}}}$

Require: Old policy $\pi_{\theta_{\text{old}}}$, old value function $V_{\phi_{\text{old}}}$, rollout length T, discount factor γ, GAE parameter λ.

for $t = 0, \ldots, T-1$ **do**

    Store transition (s_t, a_t, r_t, s_{t+1}) and old policy log-probability $\ell_t^{\text{old}} = \log \pi_{\theta_{\text{old}}}(a_t \mid s_t)$

end for

for $t = 0, \ldots, T-1$ **do**

    Compute TD residual

$$\delta_t = r_t + \gamma b_t V_{\phi_{\text{old}}}(s_{t+1}) - V_{\phi_{\text{old}}}(s_t)$$

    Set $b_t = 0$ only at a true terminal state; otherwise $b_t = 1$.

end for

$\widehat{A}_T \leftarrow 0$

for $t = T-1, \ldots, 0$ **do**

    Compute GAE advantage

$$\widehat{A}_t = \delta_t + \gamma \lambda m_t \widehat{A}_{t+1}$$

    Set $m_t = 0$ at episode or rollout boundaries; otherwise $m_t = 1$.

end for

Algorithm 2 Update the value function

Require: Value parameters ϕ, value targets $\{\widehat{R}_t\}$, minibatch $\mathcal{M}$, coefficient c_V.

1: Compute value loss

$$L^V = \frac{1}{|\mathcal{M}|} \sum_{t \in \mathcal{M}} \left(V_\phi(s_t) - \widehat{R}_t \right)^2$$

2: Detach $\widehat{R}_t$ from the computation graph.
3: Update ϕ by gradient descent on

$$\mathcal{L}^V(\phi) = c_V L^V$$

Algorithm 3 Optimize L with the clipped policy objective

Require: Policy parameters θ, old log-probabilities $\{\ell_t^{\text{old}}\}$, advantages $\{\widehat{A}_t\}$, minibatch $\mathcal{M}$, clipping parameter ϵ, coefficients c_V, c_H.

1: **for** each minibatch $\mathcal{M}$ **do**

2: Compute probability ratio

$$\rho_t(\theta) = \exp\left[\log \pi_\theta(a_t \mid s_t) - \ell_t^{\text{old}}\right]$$

3: Compute clipped policy objective

$$L^{\text{CLIP}} = \frac{1}{|\mathcal{M}|} \sum_{t \in \mathcal{M}} \min\left[\rho_t(\theta)\widehat{A}_t,\ \text{clip}(\rho_t(\theta), 1-\epsilon, 1+\epsilon)\,\widehat{A}_t\right]$$

4: Compute entropy bonus

$$L^H = \frac{1}{|\mathcal{M}|} \sum_{t \in \mathcal{M}} H(\pi_\theta(\cdot \mid s_t))$$

5: Optimize total objective by gradient descent on

$$\mathcal{L}(\theta, \phi) = -L^{\text{CLIP}} + c_V L^V - c_H L^H$$

Outline

Intro
- Measure a policy space

Moving Towards PPO
- Intro
- (2015) Trust Region Policy Optimization (TRPO)
- (2015) Generalized Advantage Estimation (GAE)
- (2016) Advantage Actor-Critic (A2C)
- (2017) Proximal Policy Optimization (PPO)

LLM fine-tuning
- Intro
- (2022) Reinforcement Learning with Human Feedback (RLHF)
- (2023) Direct Policy Optimization (DPO)
- (2024) Group Relative Policy Optimization (GRPO)
- (2024) On-Policy Distillation (OPD)

Beyond MDP
- (2021) Decision Transformer

Discussion

Intro - Policy learning for LLM

modeling the MDP world for language

LLM is essentially a policy (on a MDP?)

- state - context (prompt x + already tokens $y_{<t}$), action - next token

$$s_t = (x, y_{<t}),\ \ a_t \sim \pi(\cdot \mid s_t)$$

- transition - deterministic, just add your decided token

$$\mathbb{P}(s_{t+1} = (x, y_{<t+1}) \mid s_t = (x, y_{<t}), a_t = y_t) = 1, \quad y_{<t+1} = (y_{<t}, y_t)$$

- A trajectory?

$$\tau = (s_1, a_1, s_2, a_2, s_3, a_3, ..., s_{T-1}, a_{T-1}, s_T)$$

$$\tau = ((x), (y_1), (x, y_1), (y_2), (x, y_1, y_2), (y_3), ..., (x, y_1, ..., y_{T-1}))$$

How large is LLM's state and action space? (No way to do MCMC!)

If vocab size = 100000, max output length = 1024k, for a fix prompt, we have

- size of action space: $|\mathcal{A}|$ is 100000
- size of state space: $|\mathcal{S}|$ is 100000^{1024k}

what about $|\pi|$?

Prob of a Trajectory

Let $D^{\pi}(\tau \mid x)$ denote the probability of a trajectory τ of states on starting from prompt x and following policy π.

$$\tau = (s_1, a_1, s_2, a_2, s_3, a_3, \ldots, s_{T-1}, a_{T-1}, s_T)$$

$$\tau = ((x), (y_1), (x, y_1), (y_2), (x, y_1, y_2), (y_3), \ldots, (x, y_1, \ldots, y_{T-1}))$$

- By the Markov property and chain rule:

$$D^{\pi}(\tau \mid x) \;=\; \prod_{t=1}^{T-1} \pi(a_t \mid s_t)\, \mathbb{P}(s_{t+1} \mid s_t, a_t)$$

- Degeneration in the LLM (deterministic transition) Setting
  Let $y = (y_1, \ldots, y_{T-1})$:

$$D^{\pi}(\tau \mid x) \;=\; \prod_{t=1}^{T-1} \pi(y_t \mid x, y_{<t}) \;=\; \pi(y \mid x)$$

  so we have

~~bandit $\leftrightarrow$ per-step~~

If you take a bandit environment view, it presents a random user prompt and expects a response to the prompt.

Rethink LLM in a probability view

and what about its states/action distribution?
What LLM policy do?
-¿ generating a world
$D^{\pi}(\tau \mid x) =$

Reinforcement Learning with Human Feedback (RLHF)

A starting point (Christiano et al. 2017)(Ouyang et al. 2022)

For MDP $(\mathcal{S}, \mathcal{A}, \mathcal{R}, \mathcal{P})$, $(\mathcal{S}, \mathcal{A}, \mathcal{P})$ settled, leaving $\mathcal{R}$ undefined...

Solution of RLHF paper:

(1) Defining (Training/Estimating) a reward model
- collect comparison data (prompt+win/lose output pair (x, y_w, y_l))
- train an additional reward model $r_\phi(\cdot, \cdot)$

(2) Use the reward model to define our $\mathcal{R}$eward of World $(\mathcal{S}, \mathcal{A}, \mathcal{R}, \mathcal{P})$

$$R(s_t, a_t) = f(r_\phi(s_t, a_t))$$

DO NOT mix this two 'reward' !

(3) Then apply RL! by PPO, etc ...

choice of $r_\phi(\cdot, \cdot)$ in (Ouyang et al. 2022)'s practice
- Insight: humans are bad at scoring, but good at comparing
- a 6B model from Bradley-Terry (BT)
- critic: cannot capture non-transitive preferences

Reinforcement Learning with Human Feedback (RLHF)

A starting point (Christiano et al. 2017)(Ouyang et al. 2022)

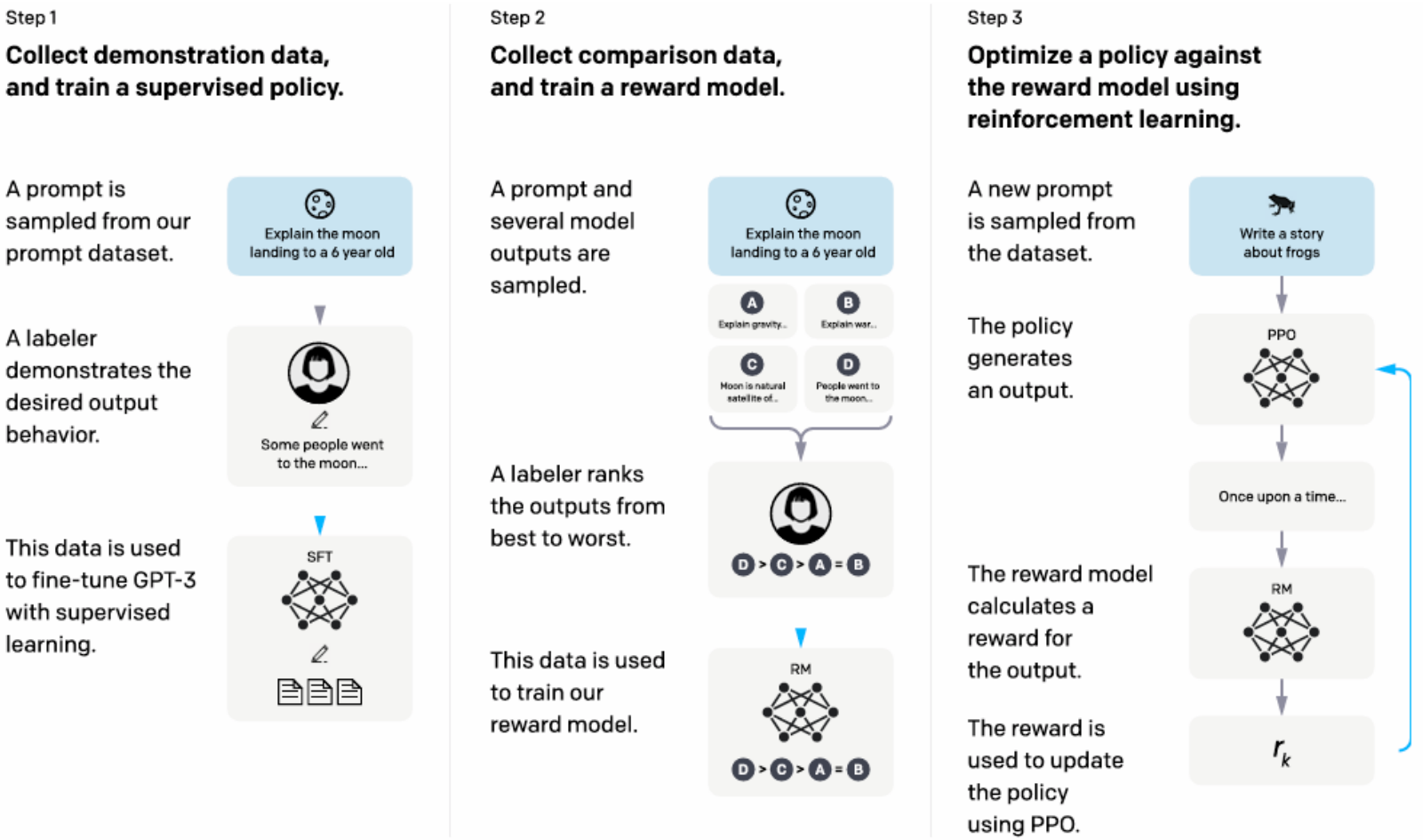


Figure 2: A diagram illustrating the three steps of our method: (1) supervised fine-tuning (SFT), (2) reward model (RM) training, and (3) reinforcement learning via proximal policy optimization (PPO) on this reward model. Blue arrows indicate that this data is used to train one of our models. In Step 2, boxes A-D are samples from our models that get ranked by labelers.

A special point of RL for LLM (diff from RL for RL)

constrained reward: Finetuning on a Base instead of Learning from Scratch

- per-token KL penalty into reward (dense reward version)

$$r_t = \begin{cases} -\beta \log \dfrac{\pi_\theta(y_t \mid x, y_{<t})}{\pi_{\text{ref}}(y_t \mid x, y_{<t})}, & t < T \\ r_\phi(x, y) - \beta \log \dfrac{\pi_\theta(y_t \mid x, y_{<t})}{\pi_{\text{ref}}(y_t \mid x, y_{<t})}, & t = T \end{cases}$$

From finite-horizon RL to the KL-regularized RLHF objective

Let $D^\pi(\cdot \mid x)$ denote the probability of a trajectory τ of states on starting from prompt x and following policy π.

$$\max_{\pi_\theta} \mathbb{E}_{x\sim\mathcal{D},\ \tau\sim D^\pi(\cdot|x)}\left[\sum_{t=0}^{T} r_t\right] \quad \text{(classic finite horizon RL goal with } \gamma = 1)$$

$$= \max_{\pi_\theta} \mathbb{E}_{x\sim\mathcal{D},\ \tau\sim D^\pi(\cdot|x)}\left[r_\phi(x, y) - \beta \sum_t \log \frac{\pi_\theta(y_t|s_t)}{\pi_{\text{ref}}(y_t|s_t)}\right]$$

$$= \max_{\pi_\theta} \mathbb{E}_{x\sim\mathcal{D},\ \tau\sim D^\pi(\cdot|x)}\left[r_\phi(x, y)\right] - \beta\, \mathbb{D}_{\text{KL}}\left(\pi_\theta(y|x) \,\|\, \pi_{\text{ref}}(y|x)\right)$$

RLHF MDP with PPO style implementation

Combining above reward and PPO formula, we get
If you stare at this function for a while, you will ...

$$\mathbb{E}_{x\sim\mathcal{D},y\sim\pi_\theta(y|x)}\left[r_\phi(x,y)\right]-\beta\mathbb{D}_{\mathrm{KL}}\left[\pi_\theta(y\mid x)\parallel\pi_{\mathrm{ref}}(y\mid x)\right]$$

corresponding PPO loss
[xx]

Direct Policy Optimization

Back to a SFT style finetuning (Rafailov et al. 2023)

Maintaining a reward function is also expensive, so 'skip' the reward function and optimize directly on offline data (x, y_w, y_l) pairs

DPO goal:

$$\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}}\left[\log\sigma\left(\beta\log\frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta\log\frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right)\right]$$

pair-wise DPO loss (sample-based)
? in what sense?

Pros and Cons of DPO easy to implement PPO-RLHF DPO online

Group Relative Policy Optimization

byebye critic, a smart MC (Shao et al. 2024)

- online RL?

Why GRPO is good for math tasks? xx

On-Policy Distillation

(R. Agarwal et al. 2024)

Outline

- Intro
  - Measure a policy space
- Moving Towards PPO
  - Intro
  - (2015) Trust Region Policy Optimization (TRPO)
  - (2015) Generalized Advantage Estimation (GAE)
  - (2016) Advantage Actor-Critic (A2C)
  - (2017) Proximal Policy Optimization (PPO)
- LLM fine-tuning
  - Intro
  - (2022) Reinforcement Learning with Human Feedback (RLHF)
  - (2023) Direct Policy Optimization (DPO)
  - (2024) Group Relative Policy Optimization (GRPO)
  - (2024) On-Policy Distillation (OPD)
- Beyond MDP
  - (2021) Decision Transformer
- Discussion

Decision Transformer

a new framework for offline RL task

Decision Transformer(DT) reformulates the offline RL task as **a sequence modeling problem**.

- the return-to-go(sum of future rewards) $\hat{R}_t = \sum_{t'=t}^{T} r_{t'}$
  - reformulated trajectory: $(\hat{R}_1, s_1, a_1, ..., \hat{R}_T, s_T, a_T)$
- Goal: learn a prediction model that predicts the action based on past returns-to-go, states, and actions(analogous to a language modeling task! action = next word)

$$\hat{a}_t = f_\theta(\hat{R}_{\leq t}, s_{\leq t}, a_{<t})$$

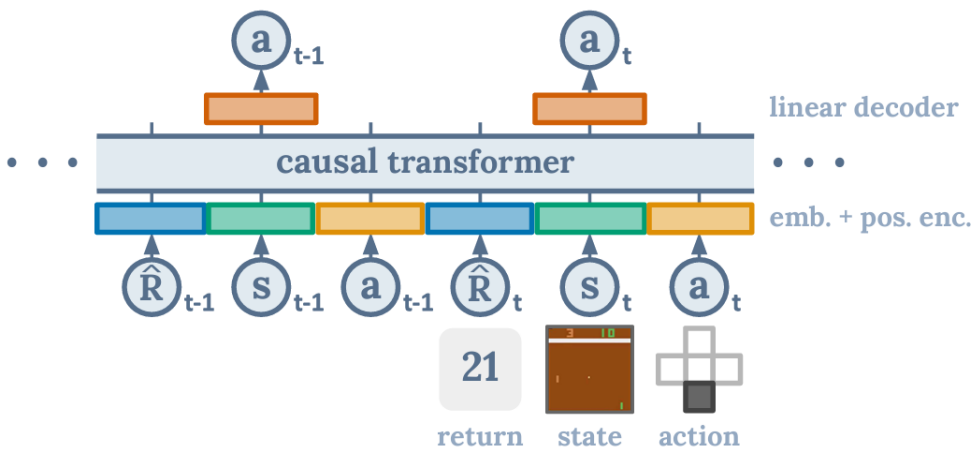


Figure: Decision Transformer architecture

Outline

- Intro
  - Measure a policy space
- Moving Towards PPO
  - Intro
  - (2015) Trust Region Policy Optimization (TRPO)
  - (2015) Generalized Advantage Estimation (GAE)
  - (2016) Advantage Actor-Critic (A2C)
  - (2017) Proximal Policy Optimization (PPO)
- LLM fine-tuning
  - Intro
  - (2022) Reinforcement Learning with Human Feedback (RLHF)
  - (2023) Direct Policy Optimization (DPO)
  - (2024) Group Relative Policy Optimization (GRPO)
  - (2024) On-Policy Distillation (OPD)
- Beyond MDP
  - (2021) Decision Transformer
- Discussion

Discussion

What does your goal like (do you need MDP assumption? is your action space continuous or discrete? finite horizon or infinite horizon or ...? do you need a deterministic policy or also good with stochastic one?)
What does your policy look like?

Optimizing a Function v.s. Optimizing a Policy

$\max f(x)$
$\max V^{\pi}(s)$

Online v.s. Offline, On-policy v.s. Off-policy

- Online RL v.s. Offline RL: can/can not interact with the environment to get new sample trajectory [?]

Estimating Advantage: MC or by Critic?

references

Agarwal, Rishabh et al. (2024). "On-policy distillation of language models: Learning from self-generated mistakes". In: *International Conference on Learning Representations*. Vol. 2024, pp. 21246–21263.

Christiano, Paul F et al. (2017). "Deep reinforcement learning from human preferences". In: *Advances in neural information processing systems* 30.

Mnih, Volodymyr et al. (2016). "Asynchronous methods for deep reinforcement learning". In: *International conference on machine learning*. PmLR, pp. 1928–1937.

Ouyang, Long et al. (2022). "Training language models to follow instructions with human feedback". In: *Advances in neural information processing systems* 35, pp. 27730–27744.

Rafailov, Rafael et al. (2023). "Direct preference optimization: Your language model is secretly a reward model". In: *Advances in neural information processing systems* 36, pp. 53728–53741.

Schulman, John, Sergey Levine, et al. (2015). "Trust region policy optimization". In: *International conference on machine learning*. PMLR, pp. 1889–1897.

Schulman, John, Philipp Moritz, et al. (2015). "High-dimensional continuous control using generalized advantage estimation". In: *arXiv preprint arXiv:1506.02438*.

Schulman, John, Filip Wolski, et al. (2017). "Proximal policy optimization algorithms". In: *arXiv preprint arXiv:1707.06347*.

Shao, Zhihong et al. (2024). "Deepseekmath: Pushing the limits of